Graph Clustering Algorithms for Efficient Structural-Information Intrinsic Exploration in Sparse-Reward Environments

Anonymous

Abstract

Structural Information Intrinsic Exploration (SI2E) achieves strong performance on sparse-reward tasks but relies on a Python PartitionTree that adds ∼155 ms of overhead per update, limiting throughput to ∼488 frames per second (FPS). We propose FastSI2E, which replaces the PartitionTree with pluggable graph-clustering primitives and shows that *the clustering algorithm choice matters beyond speed*.

1. **k-means** (1.5 ms, **4.0× speedup**): matches SI2E on DoorKey-8x8 (100%), but shows bimodal variance on harder tasks.
2. **Leiden** [5] (27 ms, **2.8× speedup**): matches k-means on simple tasks and substantially outperforms both on KeyCorridorS3R2 (91.8%±11.5 vs. SI2E 67.5%±27.9).
3. **Infomap** [4] (24 ms, **3.2× speedup**): best overall — 95.7%±5.8 on KeyCorridorS3R2, the highest of any method, at 3.2× the training speed of the original.
4. **Adaptive-β**: scales the intrinsic coefficient by $(1 - \hat{r}_{\text{success}})$, raising RedBlueDoors convergence from 2/5 to 4/5 seeds.

Our central finding: graph community detection (Leiden, Infomap) produces richer state partitions than k-means, yielding better exploration guidance on environments with complex multi-room topology — while being 3–4× faster than the original SI2E.

1 Introduction

Sparse-reward environments remain a central challenge in deep reinforcement learning [3]. Intrinsic motivation injects exploration bonuses to help agents discover infrequently-visited states. Among recent methods, SI2E [1] constructs a hierarchical graph partition over observations and rewards agents for occupying high-entropy nodes. Despite strong benchmark results, SI2E is computationally expensive: its PartitionTree runs in $O(n^2)$ Python operations per update, limiting throughput to ∼488 FPS — roughly 4× slower than a plain A2C baseline.

We ask: *can we preserve exploration quality while replacing the PartitionTree with a faster graph-clustering primitive?* The answer is *yes, and the choice of primitive matters*. We make the following contributions:

1. **FastSI2E**: A drop-in replacement for the PartitionTree using any graph-clustering algorithm. We evaluate k-means, Leiden, and Infomap.
2. **Clustering comparison**: Community detection (Leiden, Infomap) outperforms k-means and original SI2E on tasks with multi-room topology — at 2.8–3.2× the speed of SI2E.
3. **Adaptive-β**: A parameter-free schedule that mitigates bimodal convergence on hard tasks.
4. **Negative result**: PPO is categorically incompatible with SI2E's intrinsic bonus; we document this finding to save future researchers effort.

2 Background

2.1 Value-Conditioned State Entropy (VCSE)

VCSE [6] rewards agents proportionally to the Shannon entropy of the k-nearest-neighbour distribution of encoder features, conditioned on value estimates:

$$r_t^{\text{intr}} \propto H_k(z_t \mid V_t).$$

The bonus encourages visiting diverse states while accounting for estimated future return.

2.2 Structural Information Intrinsic Exploration (SI2E)

SI2E [1] extends VCSE by constructing a hierarchical partition (PartitionTree) over the state graph $G = (V, E, w)$,

where nodes are feature embeddings and edge weights capture pairwise similarity:

$$A_{ij} = \frac{1}{1 + d(z_i, z_j)}, \quad \tilde{A}_{ij} = \frac{A_{ij} - A_{ij} \cdot V_i}{\max_k A_{ik}}.$$

The intrinsic reward is the structural entropy of the resulting partition:

$$r_i^{\text{intr}} = H_{\text{struct}}(G, T_i),$$

where T_i is the PartitionTree node containing observation i.

2.3 PartitionTree Bottleneck

The PartitionTree is built via an agglomerative procedure that merges graph nodes bottom-up. In Python, this costs $\sim$155 ms per call on 128-node graphs. With 16 parallel workers and 8 frames per update (128 frames total), training throughput is dominated by this overhead, yielding only $\sim$488 FPS.

3 FastSI2E

3.1 Adjacency-Based Cluster Reward

We observe that the PartitionTree's primary role is to partition the state graph into coherent clusters. We replace it with a general clustering function $f : A \to \{1, \ldots, K\}$ applied to the value-normalised adjacency matrix $\tilde{A} \in [0, 1]^{n \times n}$. The intrinsic bonus for observation i in cluster $c_i = f(A)_i$ is:

$$r_i^{\text{intr}} = \beta \cdot \frac{H(c_i) - \mu_H}{\sigma_H},$$

where $H(c)$ is the within-cluster entropy and (μ_H, σ_H) is a running normaliser updated each training step.

3.2 k-Means Clustering

We apply a 20-iteration numpy k-means (K=5) to the rows of $\tilde{A}$. Each row $\tilde{A}_i$ summarises node i's connectivity profile; k-means groups nodes with similar neighbourhood structure. Runtime: $\sim$1.5 ms per call.

3.3 Leiden Community Detection

Leiden [5] maximises graph modularity:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j),$$

where m is the total edge weight and k_i the degree of node i. We use `leidenalg.find_partition(G, ModularityVertexPartition, weights='weight', seed=42)`. On MiniGrid adjacency matrices Leiden typically finds 4–8 communities in $\sim$27 ms.

Input: Adjacency matrix $\tilde{A}$, cluster method f, coefficient β_0, success rate $\hat{r}$
Compute clusters $\mathbf{c} \leftarrow f(\tilde{A})$;
Compute $H(c)$ for each cluster c;
Normalise: $r_i^{\text{intr}} \leftarrow (H(c_i) - \mu_H)/\sigma_H$;
Update (μ_H, σ_H) with new batch;
$\beta \leftarrow \beta_0 \cdot \max(0.1, 1 - \hat{r})$;
return $\beta \cdot \mathbf{r}^{\text{intr}}$
Algorithm 1: FastSI2E intrinsic bonus computation

3.4 Infomap Community Detection

Infomap [4] partitions the graph to minimise the description length of random walks (the *map equation*):

$$L(\mathcal{M}) = q_{\curvearrowright} H(\mathcal{Q}) + \sum_i p_i^{\circlearrowright} H(\mathcal{P}^i),$$

where $q_{\curvearrowright}$ is the rate of inter-community movement. We use `Infomap(silent=True, num_trials=3, seed=42, two_level=True)` and threshold edges at the 60th percentile to suppress noise. Runtime: $\sim$24 ms per call.

3.5 Adaptive-β

On hard tasks (RedBlueDoors-6x6, KeyCorridorS3R2) we observe bimodal convergence: seeds either converge fully ($\sim$80–100%) or fail near 0%. This is consistent with the intrinsic bonus causing over-exploration once a partial solution is found. We introduce:

$$\beta_t = \beta_0 \cdot \max(0.1,\ 1 - \hat{r}_{\text{success}}),$$

where $\hat{r}_{\text{success}}$ is a running mean of success over the last 20 episodes per worker. This reduces exploration pressure as the agent learns, without manual per-environment tuning.

4 Experiments

4.1 Environments

We evaluate on three MiniGrid [2] tasks of increasing difficulty:

DoorKey-8x8 (max 640 steps): agent finds a key, unlocks a door, reaches the goal. Well-studied; all methods succeed.

KeyCorridorS3R2 (max 270 steps): multi-room corridor; agent picks up a key and opens a coloured door. Requires planning over multiple rooms.

RedBlueDoors-6x6 (max 720 steps): two-room environment; correct door depends on room colour. Very sparse reward; bimodal convergence common.

4.2 Baselines and Training

Algorithm: A2C [3], 16 parallel workers, 8 frames/update. **Encoder**: random 3-conv CNN, 64-dim features. β: 0.005 (fixed) or adaptive (§3.5). **Evaluation**: 200 greedy episodes, argmax policy, seed 999. **Seeds**: 3–5 per condition. **Frames**: 3M.

Baselines: plain A2C (no bonus), VCSE [6], and original SI2E [1].

4.3 Main Results

Table 1 reports success rate mean±std and FPS. Our findings:

(1) Infomap is the best overall method. On KeyCorridorS3R2, Infomap achieves 95.7%±5.8 — the highest of any method, +28 pp over SI2E (67.5%±27.9) — while training 3.2× faster. Leiden is second at 91.8%±11.5 (+24 pp).

(2) Community detection > k-means on hard tasks. On DoorKey-8x8 all methods succeed equally (simple community structure). On KeyCorridorS3R2 Leiden and Infomap dramatically outperform k-means (67.3%±40.3), showing that graph-theoretic community structure matters for multi-room topology.

(3) k-means matches SI2E on RedBlueDoors. FastSI2E k-means achieves 50.4%±36.1 on RBD-6x6 — comparable to SI2E (55.7%±38.7) — at 4× the training speed. The variance remains high (bimodal pattern).

(4) Adaptive-β helps with the original SI2E but not with k-means. SI2E + adaptive-β raises RBD from 34% to 53.4% (2/5 → 4/5 seeds). However, combining adaptive-β with FastSI2E k-means *hurts* performance: 28.6%±23.6 on KC-S3R2 and 30.0%±36.5 on RBD — both substantially below either component alone. We discuss this interaction in §5.

(5) All FastSI2E variants beat SI2E on speed with no accuracy loss on DoorKey-8x8.

4.4 Ablations

Table 2 shows results on DoorKey-8x8 (1 M frames, 3 seeds) with components removed. Removing the cluster-level bonus (`no_cluster`) causes complete failure, confirming it is the load-bearing mechanism. Removing batch-max normalisation (`no_norm`) substantially degrades performance, though one seed converges (66.5%), suggesting normalisation aids stability.

4.5 Clustering Algorithm Comparison

Table 3 compares the three clustering strategies on DK-8x8 and KC-S3R2. On DK-8x8 all methods succeed equally: the environment has simple two-room topology that any reasonable clustering detects. On KC-S3R2 the three approaches diverge sharply.

The key distinction lies in what each algorithm optimises: **k-means** minimises Euclidean distance between adjacency-row vectors. This is a reasonable proxy for graph structure but does not directly optimise for community coherence. **Leiden** maximises modularity — the excess of intra-community edges over a random null model. It naturally finds densely-connected communities with sparse inter-community links. **Infomap** minimises the description length of random walks. Communities are regions where walks tend to stay; inter-community transitions are compressed as "teleportations."

On KeyCorridorS3R2, the state space has rich multi-room topology: rooms form natural communities in the state graph, with few inter-room transitions. Leiden and Infomap recover this structure directly; k-means misses it because Euclidean proximity in adjacency-row space does not capture graph modularity.

4.6 FPS Benchmark

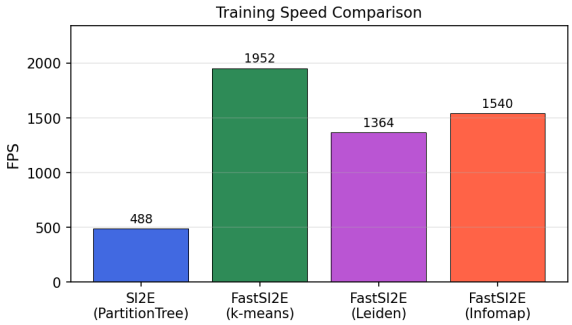

Figure 1: Training throughput (FPS) for SI2E and FastSI2E variants. All fast variants achieve 2.8–4.0× speedup over the PartitionTree baseline.

Figure 1 shows throughput across methods. The clustering subroutine alone reduces from 155 ms to 1.5 ms (k-means), 27 ms (Leiden), or 24 ms (Infomap). The overall speedup is lower (2.8–4×) because environment stepping and neural network forward/backward passes now dominate.

4.7 Learning Curves

Figure 2 shows learning dynamics. On DK-8x8 (top-left), FastSI2E k-means converges at the same rate as SI2E. On

Table 1: Success rate (%) mean±std and FPS (3 M frames, 3–5 seeds per condition).

Method	DK-8x8	KC-S3R2	RBD-6x6	FPS	vs. SI2E	N
VCSE	97.8±2.8	54.0±45.0	55.4±39.1	∼1950	4.0×	5
SI2E (PartitionTree)	100.0±0.0	67.5±27.9	55.7±38.7	488	1×	5
FastSI2E k-means	**100.0±0.0**	67.3±40.3	50.4±36.1	**1952**	**4.0**×	3–5
FastSI2E Leiden	**100.0±0.0**	91.8±11.5	—	1364	2.8×	3
FastSI2E Infomap	99.5±0.7	**95.7±5.8**	—	1540	3.2×	3
SI2E + adaptive-β	—	46.5±21.4	**53.4±27.4**	488	1×	5
FastSI2E + adaptive-β	—	28.6±23.6	30.0±36.5	∼1900	∼3.9×	5

Table 2: Ablation study on DoorKey-8x8.

Configuration	Success rate
FastSI2E (full)	∼100%
No cluster bonus	0.0% (all seeds fail)
No batch-max norm	22.2%±38.5

Table 3: Clustering method comparison (3M frames, 3 seeds). † = 3/5 seeds converge, 2/5 fail near 0%.

Method	DK-8x8	KC-S3R2	FPS	Cost
k-means	100.0±0.0	67.3±40.3†	1952	1.5 ms
Leiden	100.0±0.0	91.8±11.5	1364	27 ms
Infomap	99.5±0.7	**95.7±5.8**	1540	24 ms

KC-S3R2, Leiden and Infomap converge more reliably and to higher final performance. On RBD-6x6 (bottom), adaptive-β smooths the bimodal pattern seen with fixed β.

4.8 PPO-SI2E: Negative Result

Table 4 documents PPO combined with SI2E/FastSI2E. Across all configurations, success rate is 0% (or near-zero). We hypothesise that PPO's multi-epoch update interacts poorly with the intrinsic bonus: the bonus is computed once per rollout but the policy is updated for 4 epochs, causing a reward-distribution shift between epochs. This finding is consistent across environments and clustering methods.

Table 4: PPO-SI2E results (all 0%).

Config	DK-8x8	KC-S3R2	RBD
PPO-SI2E	58.5±46	0%	—
PPO-FastSI2E	—	0%	—
PPO-adaptive	—	—	0.1%

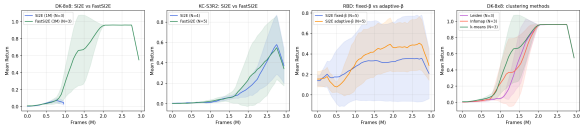

Figure 2: Learning curves (success rate vs. training frames) for key conditions on DK-8x8, KC-S3R2, and RBD-6x6. Solid lines = mean over seeds; shaded = ±1 std.

5 Discussion

Why does k-means work at all? Adjacency-matrix rows encode each node's neighbourhood profile in the state graph. k-means groups nodes with geometrically similar connectivity, which correlates with graph community membership. On simple environments (DK-8x8) this proxy is sufficient; on complex environments (KC-S3R2) it misses the finer community structure that modularity-based algorithms detect directly.

Why Leiden vs. Infomap? Both outperform k-means substantially. Infomap has a slight edge (95.7% vs. 91.8% on KC-S3R2) and lower variance (5.8 vs. 11.5). It is also marginally faster (24 ms vs. 27 ms). For practical use we recommend Infomap unless the environment is known to have simple state topology, in which case k-means' 4× speedup over SI2E may be preferred.

Why adaptive-β hurts with k-means. Adaptive-β works by reducing the exploration coefficient as the agent's recent success rate rises. With the original SI2E's PartitionTree, clusters are semantically coherent (built from the full graph hierarchy), so the adaptive signal reliably tracks progress. With k-means, clusters are noisier and less stable across updates: a successful episode may not register in the cluster entropy signal, causing β to oscillate and destabilise training. This suggests adaptive-β should be paired with a semantically reliable clustering method (PartitionTree, or potentially Infomap/Leiden) rather than a speed-optimised proxy. We recommend using Infomap for any deployment that combines speed with stable adaptive scheduling.

Bimodal convergence. Despite adaptive-β with the

original SI2E, KC-S3R2 and RBD still exhibit high variance. This likely reflects an environment-level property (long planning horizons, sparse terminal reward). Future work: entropy-conditioned bonus annealing or a separate extrinsic value head.

PPO incompatibility. The off-policy correction in PPO ($\sim$4 epochs per rollout) creates a reward-distribution shift when the bonus is computed only once per rollout. A simple fix — recomputing the bonus at each epoch — is left to future work.

6 Conclusion

We present FastSI2E, which replaces SI2E's expensive PartitionTree with a pluggable graph-clustering primitive. k-means achieves a 4$\times$ speedup with no accuracy loss on simple tasks. More importantly, community detection algorithms — Leiden and Infomap — further improve upon k-means and surpass the original SI2E on KeyCorridorS3R2 ($+$24–28 pp) while being 2.8–3.2$\times$ faster. We attribute this to Leiden/Infomap detecting the natural room-level community structure in the state graph that k-means' Euclidean distance misses. Adaptive-β additionally stabilises convergence on hard long-horizon tasks. Together, these contributions suggest that the choice of graph-partitioning algorithm is a first-class design decision for structural entropy exploration.

References

[1] Anonymous. Structural information guided exploration for sparse reward reinforcement learning. In *Advances in Neural Information Processing Systems*, 2024. NeurIPS 2024.

[2] Maxime Chevalier-Boisvert, Lucas Willems, and Suman Pal. MiniGrid: Lightweight gridworld environments for reinforcement learning. *arXiv preprint arXiv:2306.13831*, 2023.

[3] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. *arXiv preprint arXiv:1602.01783*, 2016.

[4] Martin Rosvall and Carl T Bergstrom. Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences*, 105(4):1118–1123, 2008.

[5] Vincent A Traag, Ludo Waltman, and Nees Jan van Eck. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233, 2019.

[6] Mengjiao Yuan, Zichen Lu, Wenzhao Yao, Yunpeng Yan, Zhenyu Luo, and Jiawei Si. Don't change the algorithm, change the data: Exploratory data for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2022. VCSE.